KeyKnowledgeRAG (K^2RAG): An Enhanced RAG Method for Improved LLM Question-Answering Capabilities

Hruday Markondapatnaikuni[1], Basem Suleiman[2(✉)], Shijing Chen[2], and Abdelkarim Erradi[3]

[1] University of Sydney, Sydney, Australia
smar0241@uni.sydney.edu.au
[2] University of New South Wales, Sydney, Australia
{b.suleiman,arthur.chen}@unsw.edu.au
[3] Qatar University, Doha, Qatar
erradi@qu.edu.qa

Abstract. Fine-tuning large language models (LLMs) is highly resource-intensive, making scalable alternatives like Retrieval-Augmented Generation (RAG) increasingly attractive. However, naive RAG implementations often suffer from limited retrieval precision, long processing times, and high resource costs. We propose KeyKnowledgeRAG (K^2RAG), a scalable framework that integrates dense and sparse retrieval, knowledge graphs, and summarization into a unified pipeline using a divide-and-conquer strategy. K^2RAG also includes a pre-processing step that reduces training load and enhances chunk relevance. Evaluated on the MultiHopRAG benchmark, our method achieves a mean answer similarity of **0.57** and top-quartile similarity of **0.82**, while reducing training time by **93%**, execution time by **40%**, and VRAM usage by **66%**. These results demonstrate that K^2RAG offers a scalable and efficient solution for developing lightweight, robust question-answering systems based on internal documents while offering excellent answer accuracy and quality.

Keywords: K^2RAG · RAG · Large Language Models · Retrieval Augment Generation

1 Introduction

Retrieval-Augmented Generation (RAG) systems have emerged as an efficient alternative to fine-tuning large language models (LLMs) [7]. Unlike fine-tuning, which requires modifying the base LLM through computationally expensive updates, RAG systems retrieve relevant knowledge from an external document index at runtime without altering the underlying model [1,9]. This approach not only reduces the high computational costs associated with fine-tuning but also enables dynamic knowledge updates, allowing for more flexible and up-to-date responses.

T. Taniguchi et al. (Eds.): ICONIP 2025, LNCS 16312, pp. 217–231, 2026.
https://doi.org/10.1007/978-981-95-4384-7_16

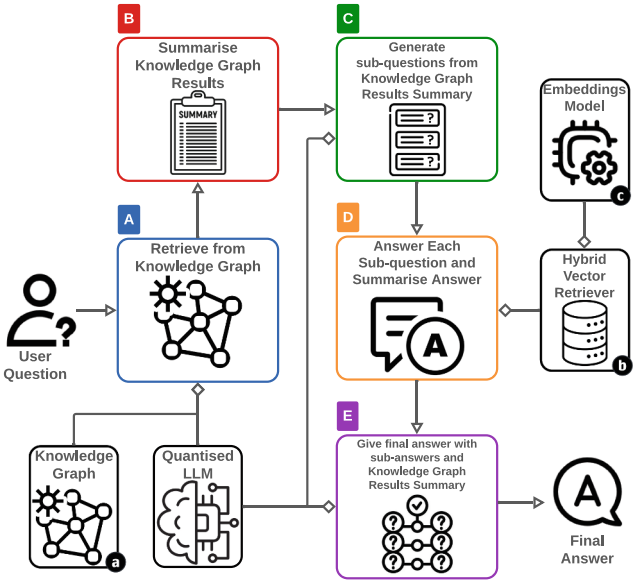

Fig. 1. K^2RAG framework overview. Component (a): Knowledge Graph. Component (b): Hybrid Retriever containing Dense and Sparse retrievers. Component (c): Embeddings Model. Step (A): Knowledge Graph results. Step (B): Summarize Knowledge Graph results. Step (C): Generating sub-questions from chunks. Step (D): Generating sub-answers for sub-questions. Step (E): Generate the final answer from sub-answers and Knowledge Graph Results Summary.

Answer Accuracy Challenges: Despite their advantages, naive RAG implementations often struggle with the "Needle-in-a-Haystack" problem [8]. In these cases, the generator model fails to extract relevant information from a vast pool of documents, leading to inaccurate responses. This issue frequently stems from suboptimal chunking and retrieval processes [18].

Chunk size plays a critical role in retrieval effectiveness. Smaller chunks may lose essential context, while larger chunks can introduce irrelevant information and unnecessarily increase the context length. Poorly optimized retrieval mechanisms further exacerbate the problem by selecting embeddings that are semantically similar but contextually irrelevant [13]. This results in the model retrieving content that appears related but does not meaningfully contribute to answering the query. Optimizing both chunking and retrieval strategies is essential to improving answer accuracy. Effective retrieval must ensure that only the most relevant information is surfaced while filtering out noise that could obscure the correct response.

Scalability Challenges: Naive RAG implementations also face scalability limitations. Most rely on 16-bit floating-point LLMs for the generation component, which significantly increases memory consumption and inference latency [19].

Deploying such models at scale becomes computationally expensive, making it crucial to explore methods that reduce memory footprint and improve efficiency.

One promising approach is the use of quantized LLMs, which require fewer computational resources and provide a more cost-effective alternative to full-precision models in addition to faster inference [19]. In an optimized RAG system, the generator LLM should primarily focus on reformatting and structuring retrieved information, rather than serving as the primary source of knowledge generation.

Beyond the LLM itself, naive RAG implementations also encounter bottlenecks in information storage components, such as Knowledge Graphs and vector databases. The training and updating of these stores can be time-consuming, leading to delays in incorporating new knowledge into the system. To address this, efficient data preprocessing techniques should be implemented to reduce corpus size while preserving key information, thus minimizing update times in production environments.

Proposed Solution - The K^2RAG Framework: To overcome these limitations, we propose K^2RAG, a framework designed to enhance both answer accuracy and scalability in RAG systems. Our approach addresses the following four research goals:

- **1. Reduce information store creation and update times:** Optimizing the training process for Knowledge Graphs and vector databases to ensure faster indexing and retrieval.
- **2. Mitigate the Needle-in-a-Haystack problem:** Optimizing the size of the context being passed to the generator LLM to promote usage of all given information to answer the question.
- **3. Improve retrieval accuracy:** Increasing the likelihood that retrieved documents contain the correct answer, reducing semantic noise.
- **4. Lower temporal and computational overhead:** Reducing the time and computation costs associated with large and full-precision models.

K^2RAG integrates concepts from existing research in a novel way to offer a more accurate and resource-efficient RAG pipeline, surpassing the performance of naive implementations.

2 Related Works

Spare and Dense Retriever RAG Methods: Hybrid retrieval approaches, such as the "Blended RAG" framework by Sawarkar et al. [16], combine six retrievers, including traditional sparse methods (e.g., BM25) and dense semantic techniques, selected based on their test performance on datasets such as Natural Questions, CORD-19, Stanford Question Answering Dataset, and HotPotQA. This method effectively retrieves relevant passages for well-defined queries (aligning with Research Goal 3), but it underperformed on vaguer queries, often returning extraneous information. For instance, evaluations on the CoQA dataset

[12] reported retrieval accuracies between 45% and 49%, partly due to an over reliance on limited document metadata.

In contrast, Mandikal et al. [11] propose a hybrid model that integrates a sparse TF-IDF retriever with a dense retriever based on a fine-tuned SPECTER2 embeddings model. By employing a weighted scoring mechanism controlled by a hyperparameter λ, their method shows that incorporating approximately 20% of the sparse score significantly improves retrieval accuracy in a medical database on Cystic Fibrosis. This content-based indexing reduces metadata dependency and partially mitigates the retrieval of irrelevant information, although challenges remain for vague queries, highlighting the need for further refinement to fully achieve Research Goals 2 and 3.

Reranker RAG Methods: To overcome the limitations of Hybrid-Search methods for vague queries (Research Goal 2), reranker methods have been developed to condense and improve the final context. This two-step process initially retrieves documents via sparse or dense techniques, followed by a reranker that sorts the results by relevance before passing them to an LLM.

For example, Michael Glass et al.'s Re^2G framework [4] integrates a BERT-based reranker, yielding improvements in recall, precision, and accuracy on benchmarks like the KILT leaderboard. However, the need to host an additional model conflicts with Research Goal 4, which aims to reduce computational and monetary costs.

Similarly, Xuegang Ma et al.'s RankLLaMA system [10] fine-tunes a LLaMA-2-7B model for reranking, achieving superior performance but relying on a resource-intensive model. This dependence further challenges the cost-efficiency objectives of Goal 4.

In summary, while rerankers help mitigate the "needle in a haystack" issue by filtering out less relevant information, their reliance on large-scale models increases computational and financial burdens, thus compromising the efficiency targets set by Research Goal 4.

Knowledge Graph RAG Methods: Knowledge graph–based approaches enhance the RAG process by organizing data based on intrinsic relationships. For example, Dawei Cheng et al. [2] demonstrated that grouping related data into sub-networks enables queries, such as "who quit Apple", to retrieve only pertinent information, rather than returning all semantically similar sentences (e.g., both "Steve Jobs quits Apple" and "David Peter leaves Starbucks"). This addresses the "needle in a haystack" problem (Goal 2) and improves retrieval rates (Goal 3).

Similarly, Diego Sanmartín's work [14] presents a Knowledge Graph RAG system that leverages LLMs to extract entity-relationship triples and employs a "Chain of Explorations" retrieval strategy. Although effective for vague queries, the method incurs high training overhead (not satisfying Goal 1) and suffers from information loss due to the compressed entity-relationship triple format. These limitations resulted in lower EM, F1, and Accuracy scores despite reducing hallucinations compared to standard semantic search.

Combining Knowledge Graphs and Vector Based Search Methods: The limitations inherent in a purely knowledge graph–based RAG framework suggest that combining the strengths of knowledge graphs with traditional semantic search methods may yield superior results. Bhaskarjit Sarmah et al. [15] propose such an integrated approach through their HybridRAG framework. This framework concatenates outputs from a knowledge graph query, with content indexed by extracting entity relationships in chunk sizes of 1024, with results from a semantic search, which similarly employs chunk sizes of 1024 and the text-embedding-ada-002 model. The aggregated context is then provided to GPT-3.5 to generate the final answer. The dataset for this study was constructed by web scraping earnings call documents of companies listed on the NIFTY 50, and evaluation was performed using the RAGAS evaluation platform [3].

The HybridRAG framework demonstrated improvements in metrics such as Faithfulness, Answer Relevancy Score, and Context Recall. These enhancements support Goal 2 by reducing the "needle in a haystack" problem and Goal 3 by increasing the rate of retrieving relevant passages for answering questions. However, the framework underperformed in Context Precision when compared to standalone implementations of both knowledge graph–based and vector database–based RAG systems. This shortcoming indicates that the current integration does not fully leverage the advantages of traditional vector search techniques, particularly hybrid search techniques, which is critical for fully meeting Goal 3.

3 Methodology

3.1 K^2RAG

To meet the research goals, the K^2RAG framework implements four techniques:

- **Knowledge Graph:** K^2RAG employs a knowledge graph [6] component to organize and interconnect topics within the corpus [Fig. 1(a)]. Unlike traditional chunking approaches, which struggle to balance the trade-off between providing sufficient context and avoiding irrelevant information, the knowledge graph addresses this issue by structuring content into interconnected nodes which enables knowledge-rich and more context-aware retrieval capable of handling complex or vague queries that vector-based search methods often struggle with.
- **Hybrid Search:** To minimize false positive embeddings—where semantically similar but irrelevant chunks are retrieved—K^2RAG integrates a hybrid retriever [Fig. 1(b)]. Inspired by Mandikal et al.'s paper [11], we weight dense vector and sparse vector retrieval at an optimized 80%/20% ratio. This hybrid method significantly improves retrieval accuracy, helping reduce noise by ensuring the most relevant content is retrieved.
- **Summarization:** To combat the "Needle-in-Haystack" issue, K^2RAG incorporates summarization at multiple stages in the pipeline [Fig. 1 (B) and (E)]. To speed up indexing, documents are summarized before being indexed into

vector and knowledge graph stores. Additionally retrieved content is summarized at query time to further refine the context provided to the LLM. By summarizing at both indexing and retrieval, we ensure concise, high-quality input for generation.

– **Lightweight Models:** For VRAM resources efficiency, K^2RAG leverages a lightweight Longformer-based text summarizer [5], along with quantized LLMs to produce a low VRAM and resource-efficient pipeline without compromising output quality.

Corpus Summarization Process: To address research goal 1, we decrease training times of the K^2RAG framework compared to naive RAG frameworks by summarizing the training corpus to reduce its size while preserving as much information as possible. Hence a Longformer based model [5] fine tuned for long text summarization is used as it's self-attention mechanism is highly optimized for capturing and processing large chunks of texts compared to those seen in traditional transformer models. This helps speed up training times of information store components used in K^2RAG. To create our summarized corpus we iterate over each article in the corpus and then summarize it using the loaded in Longformer summarizer model. Then these results are stored into a CSV format from which then they can be loaded in for training the RAG pipeline.

Indexing: The chunking strategy to generate a set of chunks C_d from a single document or text d is defined as:

$$C_d = \bigcup_{i=1}^{L_t} \left\{ \left\{ x_{((i-1)*(S-O))+j} \mid j = 1, 2, \ldots, S \right\} \right\}, \; L_t = \left\lfloor \frac{Nt_d - O}{S - O} \right\rfloor \tag{1}$$

where S is the desired chunk size, O is the desired chunk overlap, Nt_d is the total number of tokens in the document d. The inner set denotes a chunk constructed from tokens $x_{((i-1)*(S-O))+1}$ to $x_{((i-1)*(S-O))+S}$ and L_t is the limit of tokens to process to ensure the number of chunks of size S and overlap O are maximized.

We then apply the chunking strategy defined in (1) across all documents in the summarized corpus cor_{sum} and take the union of chunks to give us the final set of chunks C to pass to the knowledge graph and vector stores for indexing:

$$C = \bigcup_{d=1}^{|cor_{sum}|} C_d \mid d \in cor_{sum} \tag{2}$$

The configuration used to train all information retrievers is $S = 256, O = 20$ when chunking for the dense and sparse vector stores indexing, and $S = 300, O = 100$ when chunking for knowledge graph indexing.

This was decided in accordance with Xiaohua Wang et al.'s paper for vector stores indexing which showed 512 and 256 token chunks achieved the best results in Faithfulness and Relevancy metrics [18]. Chunk size less than 512 is used to

help promote having smaller contexts generated to pass to the generator LLM to address the "Needle-in-Haystack" challenge [8].

Retrieval and Generation: We establish the top k chunks C_q retrieved from the dense and sparse vector databases for a question q is a **hybrid retriever** [Fig. 1 (b)] defined as:

$$C_q = \bigcup_{i=1}^{k} \{c \mid scr(q, c, \lambda) = \max_i[S]\}, \; S = \{scr(q, c, \lambda) \mid c \in C\} \tag{3}$$

where S is the set of scores computed for each chunk in C (2) using a scoring function scr of the question q, chunk c and a weighting parameter λ:

$$scr(q, c, \lambda) = \lambda \frac{e(q) \cdot e(c)}{|e(q)| * |e(c)|} + (1 - \lambda) \frac{z(q) \cdot z(c)}{|z(q)| * |z(c)|} \tag{4}$$

where e represents the dense embeddings model as a function and z represents the sparse embedding function.

The first step in the K^2RAG pipeline [Fig. 1] is to use a knowledge graph trained on summarized data to generate an answer containing the identified topics relevant to the query [Fig. 1(A)]. Then a Longformer model is used to summarize the results obtained from the knowledge graph [Fig. 1(B)]. Over the knowledge graph results summary, we split it into chunks of $S = 128$ and $O = 10$ following our text chunking strategy (1). This is so that each chunk contains information about a clear topic needed to help answer the question from which for each chunk we use a quantized LLM to create a sub-question out of it [Fig. 1(C)]. The sub-question is then used to query the dense and sparse vector databases using the hybrid retriever with $\lambda = 0.8$ [11] and $k = 10$ (4) [Fig. 1(D)] where an embeddings model is used when querying the dense vector database [Fig. 1(c)].

We then use the quantized LLM with the results from the hybrid retriever as context to generate the answer to the sub-question and then store into a list containing all the sub-answers. Once completed for each chunk, we then concatenate all the sub-answers and use the Longformer model to summarize the results so that we can reduce the size of the context [Fig. 1(D)]. Finally, the summarized sub-answers and the summarized knowledge graph results are concatenated and passed as context to the quantized LLM to generate the final answer to the question [Fig. 1(E)].

3.2 Implemented Naive RAG Pipelines

We implemented 4 common naive RAG pipelines; **Semantic** - Retrieving only from a Dense Vector Database. **Keyword** - Retrieving only from a Sparse Vector Database. **Hybrid** - Retrieving from both Dense and Sparse Vector Database. **Knowledge Graph (KG)** - Retrieving only from a Knowledge Graph.

Indexing: The indexing process for each information store is the same as in K^2RAG but with chunks C created with the unsummarized corpus cor_{unsum} instead:

$$C = \bigcup_{d=1}^{|cor_{unsum}|} C_d \mid d \in cor_{unsum} \tag{5}$$

Retrieval and Generation: All naive pipelines follow the same process where information related to the question is retrieved and then passed to an FP16 LLM to generate the answer. For our Semantic, Keyword, and Hybrid pipelines we used a hybrid retriever with weighting of $\lambda = 1$, $\lambda = 0$ and $\lambda = 0.8$ respectively.

3.3 Evaluation Framework

Dataset Selection and Description: In order to perform a comprehensive evaluation on the proposed framework the dataset must contain question answer pairs derived from a document corpus along with the corpus itself in plain text format where answers are to be assumed as the ground truth to the questions. Due to time and resource constraints we were unable to create our own dataset from scratch, hence the MultiHop-RAG dataset [17] was used as it satisfies the aforementioned requirements.

The MultiHop-RAG dataset contains corpus data which was used for training along with a test dataset containing 2555 question answer pairs from which we performed evaluation of the pipeline. The corpus contains 609 articles covering various topics such as entertainment, health, sports, and technology [17].

Evaluation Process: We used a K-fold evaluation framework where we split the data into K=10 folds. For each pipeline, we generated the answers for each question in each fold and captured answering times for the questions to compare the execution times.

To evaluate the accuracy of a pipeline's answer, we computed the similarity between the ground truth and the pipeline's output as a scoring function s of the plaintext ground truth and pipeline output for each question defined as:

$$s(o,t) = \frac{e(o) \cdot e(t)}{|e(o)| * |e(t)|}, \tag{6}$$

where o is the pipeline's output, t is the ground truth and e represents the embeddings model as a function to generate the vector embedding of a plaintext input.

3.4 Practical Implementation:

The Corpus Summarization and Evaluation processes were performed on a Linux machine with an NVIDIA L4 24GB VRAM GPU and 32GB RAM. We used the following tech stack for implementing all pipelines; **Embeddings Model:** *nomic-embed-text*, **Quantised LLM:** *Mistral-7B-Q4*, **FP16 LLM:** *Mistral-7B-FP16*, **Summariser Model:** *pszemraj/led-base-book-summary from HuggingFace*, **Sparse Vector Database:** *Okapi BM25*,

Dense Vector Database: *ChromaDB*, **Knowledge Graph Implementation:** *Microsoft GraphRAG with Mistral-7B-Q4 as entity-relationship extractor and output generator.*

4 Results

Corpus Summarization: Corpus summarization took roughly **25 min** to complete. Most documents have been reduced by roughly **86%** to **92%** while the overall mean reduction is roughly **89%** [Fig. 2]. We attribute the consistent large reduction in size of the documents to considerable white space removed in many of the documents, leading to an effective reduction.

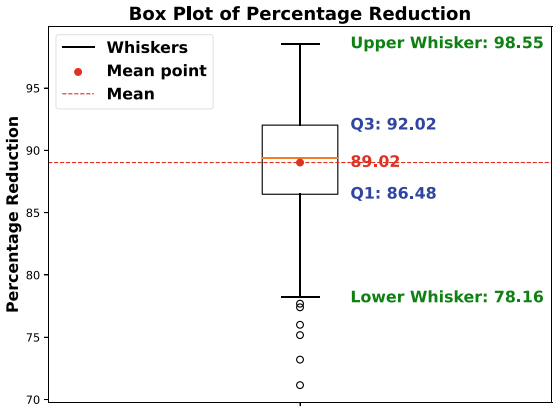

Fig. 2. Boxplot of corpus size reduction.

Training Times: By performing corpus summarization, sparse and dense vector stores, and knowledge graph creation times reduced by **89%**, **97%** and **94%** respectively with significant time saved for knowledge graph creation from 18 h to 1 h [Fig. 3] [Table 1]. Hence, in exchange for **25 min** of performing summarization, training times were reduced on average by **93%** across all information stores, successfully achieving Research Goal 1 in reducing creation times of information store components, particularly the knowledge graph.

Answer Accuracy: We observe that the score distributions for the Semantic, Keyword, Hybrid Search, and KG pipelines are very similar [Fig. 4]. This suggests that although these naive pipelines utilize different components, they cannot fully exploit their potential for Question Answering, which is why they are considered naive. In contrast, the distribution for our K^2RAG framework demonstrates a significantly larger Q_3 quartile compared to the naive frameworks, which indicates that K^2RAG was able to answer more questions with

Table 1. Table of Training Times

Information Store	Training Times
Dense Vector Full Corpus	4010s
Dense Vector Summarized Corpus	**441s**
Sparse Vector Full Corpus	356s
Sparse Vector Summarized Corpus	**12s**
Knowledge Graph Full Corpus	64785s
Knowledge Graph Summarized Corpus	**3915s**

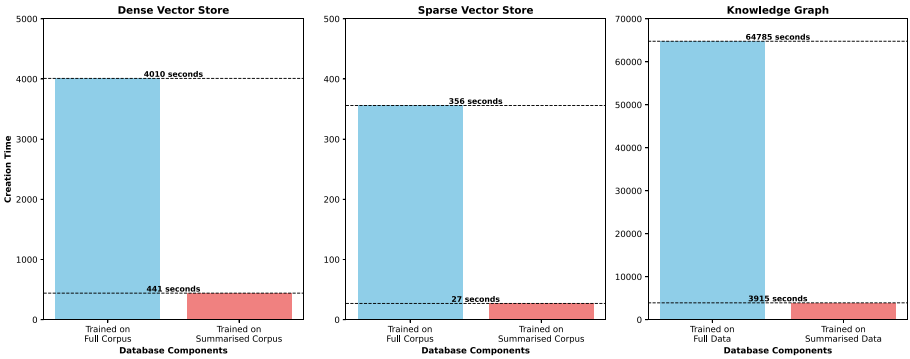

Fig. 3. Information store component training times with summarized and unsummarized corpus. Training on the summarized corpus took significantly less time compared to unsummarized corpus.

greater similarity to the ground truths. K^2RAG has a slightly higher mean accuracy of **0.57** compared to other naive methods despite a high Q_3 quartile value [Table 2]. This implies that K^2RAG is better than naive RAG pipelines in answering certain types of questions which pushed the Q_3 quartile value up, but there were not enough of these types of questions to induce a greater increase in the mean similarity. Overall, K^2RAG achieved the highest mean similarity score of **0.57** and the top Q_3 quartile score of **0.82** [Table 2], successfully achieving Research Goals 2 and 3 by retrieving only the most relevant information and creating better contexts for generating accurate and information-rich answers.

Execution Times: Semantic Search, Keyword Search, and Hybrid Search were the fastest running pipelines. KG Search recorded the longest mean runtime at 117.31 s, with a small interquartile range, indicating consistent performance [Fig. 5]. The longer runtime can be attributed to the large, unsummarized knowledge graph used in retrieval. Our proposed K^2RAG framework achieved a mean runtime of 70.25 s, significantly faster than naive KG Search but slower than vector based search methods such as semantic, keyword and hybrid search [Table 3]. This is expected as querying from the knowledge graph component takes more

Table 2. Table of Pipeline Answer K^2RAG achieved the best performance with highest mean and 3rd Quartile value.

Pipeline	Q_1	Mean	Q_3
Naive Semantic Search	0.40	0.55	0.67
Naive Keyword Search	**0.41**	0.55	0.67
Naive Hybrid Search	0.4	0.56	0.67
Naive Knowledge Graph Search	0.4	0.54	0.4
K^2RAG	0.4	**0.57**	**0.82**

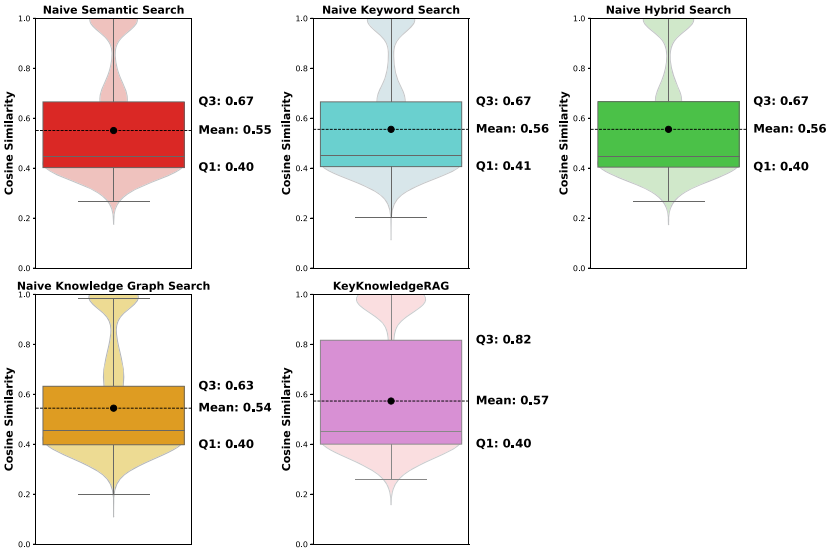

Fig. 4. Pipeline Answer Accuracy results. K^2RAG achieved the best performance with highest mean and 3rd Quartile value.

time as it is not just a simple retrieval based on matching vectors. K^2RAG exhibited a wider inter-quartile range due to the sub-questions step, which increases the number of chunks as the size of the knowledge graph results grows. Despite having more steps, K^2RAG had faster execution times than KG Search and hence successfully achieving Research Goal 4 with faster responses.

Memory Footprint: The memory footprint analysis, as presented in Table 4, highlights the significant reduction in VRAM consumption achieved by K^2RAG compared to other retrieval pipelines. Specifically, while the Semantic Search Pipeline, Keyword Search Pipeline, and Hybrid Search Pipeline each require 14.3GB of memory, and the Knowledge Graph Search demands an even higher 18.1GB. Meanwhile K^2RAG operates with only 5GB of VRAM [Table 4].

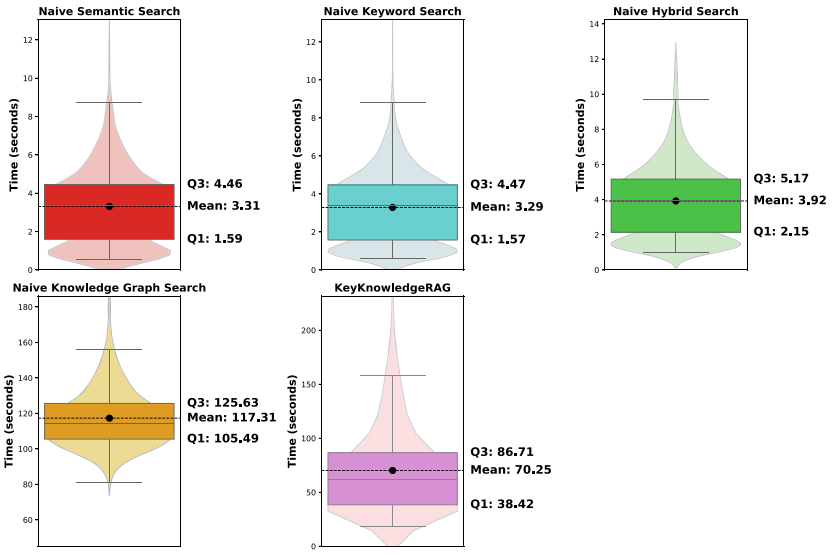

Fig. 5. Pipeline Execution Times results. K^2RAG is faster than KG Search despite more steps.

Table 3. Table of Execution Times. K^2RAG is faster than KG Search despite more steps.

Pipeline	**Mean Execution Time**
Naive Semantic Search	3.31s
Naive Keyword Search	**3.29s**
Naive Hybrid Search	3.92s
Naive Knowledge Graph Search	117.31s
K^2RAG	70.25s

This reduction represents a mean threefold decrease in memory usage relative to the other pipelines, demonstrating a substantial optimization in resource efficiency. Such an improvement directly aligns with Research Goal 4, which emphasizes minimizing computational resource requirements.

5 Discussion

Research Goal 1 - Reduce Information Store Creation and Update Times: K^2RAG effectively reduces the creation times for Knowledge Graph and Sparse and Dense vector databases, such as the Semantic and BM25-based Keyword databases. Performing text summarization on the training corpus led

Table 4. Table of Memory Footprints

Pipeline	Memory Footprint
Semantic Search Pipeline	14.3GB
Keyword Search Pipeline	14.3GB
Hybrid Search Pipeline	14.3GB
Knowledge Graph Search	18.1GB
KeyKnowledgeRAG	**5GB**

to an average reduction in training times of up to **93%**, with only **25 min** spent on summarization.

For instance, the Semantic database creation, which originally took **4010 s (67 min)**, was reduced to **441 s (7.35 min)** when using the summarized corpus. Even when accounting for the summarization process, the total time was **32.25 min**, significantly lower than training with the full corpus. Similarly, Knowledge Graph creation time dropped from nearly **18 h** to around **1 h**, thanks to the reuse of the summarized corpus. This approach not only saves time and resources but also reduces the cost of LLM-based Knowledge Graph creation, which is typically expensive to host.

Research Goal 2 - Mitigate the Needle-in-a-Haystack Problem: K^2RAG effectively mitigates the "Needle in a Haystack" problem, which can hinder LLM performance when handling large contexts. By integrating the Longformer-based *led-base-book-summary* summarizer at key stages of the retrieval pipeline, the framework reduces context size while preserving essential information.

K^2RAG maintained answer quality and even outperformed naive pipelines on certain questions, as reflected in the higher accuracy and Q_3 quantile. This approach allows for improved LLM performance without a bloated context, preventing the model from missing critical details and enhancing the final answer quality.

Research Goal 3 - Improve Retrieval Accuracy: The K^2RAG framework enhances passage retrieval through two key components. First, the knowledge component identifies relevant topics for answering the question. Based on these results, the system further explores each topic within the knowledge graph to extract more concrete and relevant chunks. This approach enables the development of more comprehensive answers. Similar to Research Goal 2, this strategy improves the quality of the final LLM-generated responses.

Research Goal 4 - Lower Temporal and Computational Overhead: While rerankers enhance passage retrieval accuracy, they require an additional large model, increasing resource demands. Similarly, the naive Knowledge Graph RAG pipeline depends on two LLMs: *Mistral-7B-FP16* (14GB VRAM) and *Mistral-7B-Q4* (4.1GB VRAM).

In contrast, the K^2RAG framework streamlines the process by utilizing only *Mistral-7B-Q4* (4.1GB VRAM) for querying the Knowledge Graph, question generation, and final answer generation, along with a small summarization model requiring just 648MB VRAM. This results in a total VRAM usage of 4.8GB, allowing for efficient RAG pipelines on smaller machines while significantly reducing computational and monetary costs.

Moreover, K^2RAG achieves faster response times and superior answer quality compared to naive RAG pipelines. This is evident in its reduced question-answering times relative to the naive Knowledge Graph Search RAG, enabling K^2RAG to process more queries in a span of time.

Limitations: While our RAG framework achieves a strong top-quartile similarity score ($Q_3 = 0.82$), the mean similarity of 0.57 suggests that performance gains are concentrated in specific question types, such as those requiring multi-hop reasoning or rich entity context. This highlights a potential area for improvement in handling broader or more diverse queries. Additionally, the use of broad knowledge graph traversal, while enhancing coverage, may occasionally introduce noise that limits precision. Future work could explore more targeted subgraph retrieval or adaptive KG filtering to further boost accuracy across a wider question set.

6 Conclusion

This paper introduces the K^2RAG framework, which addresses key limitations of traditional RAG methods. By utilizing a summarized corpus, we achieved an average reduction of 93% in training times. When combined with quantized models and an advanced retrieval process, K^2RAG outperformed all tested naive RAG methods in terms of answer accuracy, resource efficiency, and execution speed, even with more complex steps. As a result, K^2RAG offers a more accurate and lightweight RAG pipeline. While the framework demonstrated a slight improvement in mean similarity and a higher Q_3 value, it primarily excelled in answering certain question types. For future work, we recommend testing K^2RAG on more datasets to assess its performance on a wider range of topics, and enhancing knowledge graph retrieval techniques to further improve answer accuracy across all question types by identifying a broader set of relevant topics.

References

1. Alselwi, G., et al.: Long context modeling with ranked memory-augmented retrieval. arXiv preprint arXiv:2503.14800 (2025)
2. Cheng, D., Yang, F., Xiang, S., Liu, J.: Financial time series forecasting with multi-modality graph neural network. Pattern Recogn. **121**, 108218 (2022)
3. Es, S., James, J., Espinosa-Anke, L., Schockaert, S.: Ragas: automated evaluation of retrieval augmented generation (2023). https://arxiv.org/abs/2309.15217
4. Glass, M., Rossiello, G., Chowdhury, M.F.M., Naik, A., Cai, P., Gliozzo, A.: Re2G: Retrieve, rerank, generate. In: Carpuat, M., de Marneffe, M.C., Meza Ruiz, I.V. (eds.) Proceedings of the 2022 Conference of the North American Chapter of

the Association for Computational Linguistics: Human Language Technologies, pp. 2701–2715. Association for Computational Linguistics, Seattle, United States (2022)
5. Guo, M., et al.: LongT5: Efficient text-to-text transformer for long sequences. In: Carpuat, M., de Marneffe, M.C., Meza Ruiz, I.V. (eds.) Findings of the Association for Computational Linguistics: NAACL 2022, pp. 724–736. Association for Computational Linguistics, Seattle, United States (2022)
6. Hogan, A., et al.: Knowledge graphs. ACM Comput. Surv. **54**(4), 1–37 (2021)
7. Huq, S.M., Suleiman, B.: Content filtering on YouTube: an LLM approach for detecting and scoring harmful content. In: Companion Proceedings of the ACM on Web Conference 2025, pp. 1988–1992 (2025)
8. Laban, P., Fabbri, A.R., Xiong, C., Wu, C.S.: Summary of a haystack: a challenge to long-context LLMs and rag systems (2024). https://arxiv.org/abs/2407.01370
9. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (eds.) Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474. Curran Associates, Inc. (2020)
10. Ma, X., Wang, L., Yang, N., Wei, F., Lin, J.: Fine-tuning llama for multi-stage text retrieval. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, pp. 2421–2425. Association for Computing Machinery, New York (2024)
11. Mandikal, P., Mooney, R.: Sparse meets dense: a hybrid approach to enhance scientific document retrieval. CoRR abs/2401.04055 (2024). https://doi.org/10.48550/ARXIV.2401.04055
12. Reddy, S., Chen, D., Manning, C.D.: CoQA: a conversational question answering challenge. Trans. Assoc. Comput. Linguist. **7**, 249–266 (2019)
13. Rossi, N., et al.: Relevance filtering for embedding-based retrieval. In: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, pp. 4828–4835. Association for Computing Machinery, New York (2024)
14. Sanmartin, D.: Kg-rag: bridging the gap between knowledge and creativity (2024). https://arxiv.org/abs/2405.12035
15. Sarmah, B., Mehta, D., Hall, B., Rao, R., Patel, S., Pasquali, S.: Hybridrag: integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. In: Proceedings of the 5th ACM International Conference on AI in Finance, ICAIF 2024, pp. 608–616. Association for Computing Machinery, New York (2024)
16. Sawarkar, K., Mangal, A., Solanki, S.R.: Blended rag: improving rag (retriever-augmented generation) accuracy with semantic search and hybrid query-based retrievers. In: 2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR), pp. 155–161 (2024)
17. Tang, Y., Yang, Y.: Multihop-RAG: benchmarking retrieval-augmented generation for multi-hop queries. In: First Conference on Language Modeling (2024)
18. Wang, X., et al.: Searching for best practices in retrieval-augmented generation (2024). https://arxiv.org/abs/2407.01219
19. Yang, S., et al.: Retrieval-augmented generation with quantized large language models: a comparative analysis. In: Proceedings of the 2023 5th International Conference on Internet of Things, Automation and Artificial Intelligence, IoTAAI 2023, pp. 120–124. Association for Computing Machinery, New York (2024)